Tabla PARAMETROS

Define los parámetros válidos que pueden ser utilizados para configurar las terminales.

Campo	Tipo	Obligatorio	Descripción
NOMBRE_PARAMETRO	VARCHAR2(100 CHAR)	Sí	Identificador único del parámetro. Es la clave primaria de la tabla. Ej.: ORIGEN, COBERTURAS_ADMITIDAS.
DESCRIPCION	VARCHAR2(4000 CHAR)	Sí	Descripción funcional del parámetro y de su finalidad.
TIPO_DATO	VARCHAR2(20 CHAR)	Sí	Tipo de dato almacenado. Valores admitidos: STRING, NUMBER, BOOLEAN, JSON.
MULTIPLE	CHAR(1 CHAR)	Sí	Indica si el parámetro admite múltiples valores. Valores posibles: S o N.
ORIGEN_DATOS	VARCHAR2(100 CHAR)	No	Identifica el origen de los valores disponibles cuando estos deben seleccionarse de un conjunto determinado. Ej.: COBERTURAS. Si es NULL, el valor no depende de un origen de datos.
CDATE	DATE	No	Fecha de creación del registro.

Basic Image Generation

```
<Template data={{
    API_KEY_REF,
    MODEL: 'google/gemini-2.5-flash-image'
}}>
.
const openRouter = new OpenRouter({ apiKey: '{{API_KEY_REF}}' });
const result = await openRouter.chat.send({ model: '{{MODEL}}', messages: [{ role:
 'user', content: 'Generate a beautiful sunset over mountains with cinematic
lighting and reflections on a lake' }], modalities: ['image', 'text'] });
const images = result.choices[0].message.images ?? [];
for (const [index, image] of images.entries()) {
    const imageUrl = image.image_url.url;
    console.log(`Generated image ${index + 1}: ${imageUrl.substring(0, 50)}...`);
}
.
```

Supported aspect ratios

The section following a long code sample must start below it without overlap.

Multi-page code sample

```
const generatedValue1 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 1) });
const generatedValue2 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 2) });
const generatedValue3 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 3) });
const generatedValue4 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 4) });
const generatedValue5 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 5) });
const generatedValue6 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 6) });
const generatedValue7 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 7) });
const generatedValue8 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 8) });
const generatedValue9 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 9) });
const generatedValue10 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 10) });
const generatedValue11 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 11) });
const generatedValue12 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 12) });
const generatedValue13 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 13) });
const generatedValue14 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 14) });
const generatedValue15 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 15) });
const generatedValue16 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 16) });
const generatedValue17 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 17) });
const generatedValue18 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 18) });
const generatedValue19 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 19) });
const generatedValue20 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 20) });
const generatedValue21 = await client.responses.create({ model: selectedModel,
```

```
input: buildDetailedRequest(sourceDocument, userPreferences, 21) });
const generatedValue22 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 22) });
const generatedValue23 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 23) });
const generatedValue24 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 24) });
const generatedValue25 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 25) });
const generatedValue26 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 26) });
const generatedValue27 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 27) });
const generatedValue28 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 28) });
const generatedValue29 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 29) });
const generatedValue30 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 30) });
const generatedValue31 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 31) });
const generatedValue32 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 32) });
const generatedValue33 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 33) });
const generatedValue34 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 34) });
const generatedValue35 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 35) });
const generatedValue36 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 36) });
const generatedValue37 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 37) });
const generatedValue38 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 38) });
const generatedValue39 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 39) });
const generatedValue40 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 40) });
const generatedValue41 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 41) });
const generatedValue42 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 42) });
const generatedValue43 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 43) });
const generatedValue44 = await client.responses.create({ model: selectedModel,
```

```
input: buildDetailedRequest(sourceDocument, userPreferences, 44) });
const generatedValue45 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 45) });
const generatedValue46 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 46) });
const generatedValue47 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 47) });
const generatedValue48 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 48) });
const generatedValue49 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 49) });
const generatedValue50 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 50) });
const generatedValue51 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 51) });
const generatedValue52 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 52) });
const generatedValue53 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 53) });
const generatedValue54 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 54) });
const generatedValue55 = await client.responses.create({ model: selectedModel,
input: buildDetailedRequest(sourceDocument, userPreferences, 55) });
.
```

Content after multi-page code

This heading and paragraph must appear after the final code line, never over it.

Section 1

This paragraph exercises pagination with enough natural text to wrap across the available line width. It verifies that body content remains readable and that headings stay with the paragraph that follows them.

> A compact callout should stay together on a page whenever space allows.

- A concise item with useful detail
- Another item that should never overlap surrounding content

Section 2

This paragraph exercises pagination with enough natural text to wrap across the available line width. It verifies that body content remains readable and that headings stay with the paragraph that follows them.

> A compact callout should stay together on a page whenever space allows.

- A concise item with useful detail
- Another item that should never overlap surrounding content

Section 3

This paragraph exercises pagination with enough natural text to wrap across the available line width. It verifies that body content remains readable and that headings stay with the paragraph that follows them.

> A compact callout should stay together on a page whenever space allows.

- A concise item with useful detail
- Another item that should never overlap surrounding content

Section 4

This paragraph exercises pagination with enough natural text to wrap across the available line width. It verifies that body content remains readable and that headings stay with the paragraph that follows them.

> A compact callout should stay together on a page whenever space allows.

- A concise item with useful detail
- Another item that should never overlap surrounding content